

Algorithms & Data Structures

Week 02

Ahmet Feyzi Ateş

General Information (1st Phase)

- Data, memory, and CPU relationship
- Algorithmic thinking and problem-solving approach
- How search and sort algorithms work
- Time and memory complexity of algorithms
- Solution analysis with **Big-O** notation
- Evaluating and improving code performance
- Stack, Queue, and Hash Table logic
- Recursion and recursive thinking
- Introduction to Dynamic Programming
- *Practical examples from different programming languages*

First Algorithm Example: Searching

1. **Problem:** Given a sequence of items, determine whether a given item is present in the sequence, and if so, find its position (index).
2. We want an exact solution.
3. The appropriate data structure for this problem is **array**.
4. Here is the pseudocode for an algorithm to solve the problem:

ALGORITHM *SequentialSearch*($A[0..n - 1]$, K)

//Searches for a given value in a given array by sequential search

//Input: An array $A[0..n - 1]$ and a search key K

//Output: The index of the first element of A that matches K

// or -1 if there are no matching elements

$i \leftarrow 0$

while $i < n$ **and** $A[i] \neq K$ **do**

$i \leftarrow i + 1$

if $i < n$ **return** i

else return -1

First Algorithm Example: Searching

6. Analyze the algorithm.

- How many iterations of the while loop is needed for the algorithm to solve the problem and return (or stop)?

or

- How many comparison operations must be executed for the algorithm to solve the problem and return (or stop)?

- **Worst case** (the item is not present in the sequence or it is the last item in the sequence):
 - **$n-1$ iterations of the while loop or n comparisons**
- **Best case** (the item is the first item in the sequence)
 - **0 iterations or 2 comparisons**
- **Average case** (the item can be anywhere) $\approx n/2$
 - **depends on assumptions about the probabilistic distribution of input data, whether duplicates are allowed or not, etc.**

Second Algorithm Example: Binary Search

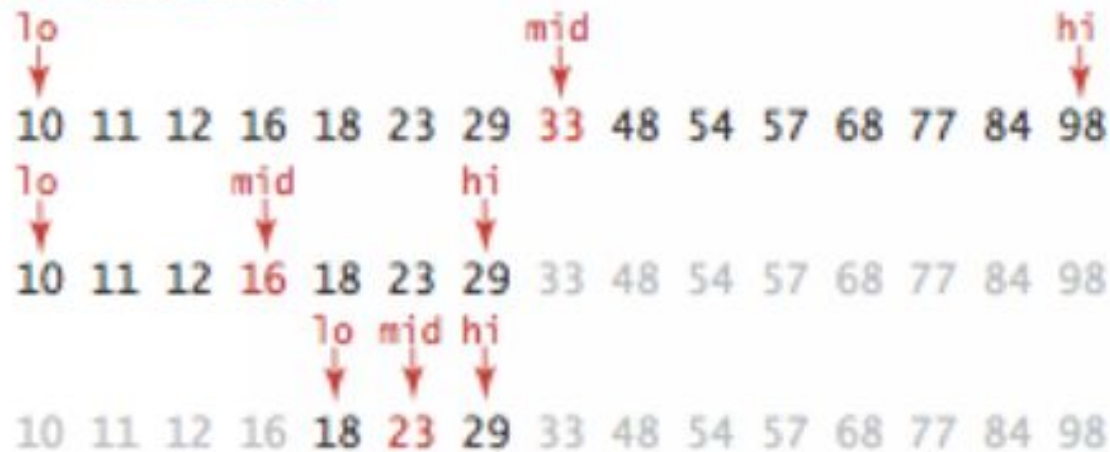
How the things can change if we already know that the input is already in sorted form (in descending or ascending order).

1. **Problem:** Given a sequence of items in sorted form (ascending), determine whether a given item is present in the sequence, and if so, find its position (index).
2. We want an exact solution.
3. The appropriate data structure for this problem is **array**.
4. Here is the pseudocode for an algorithm to solve the problem:

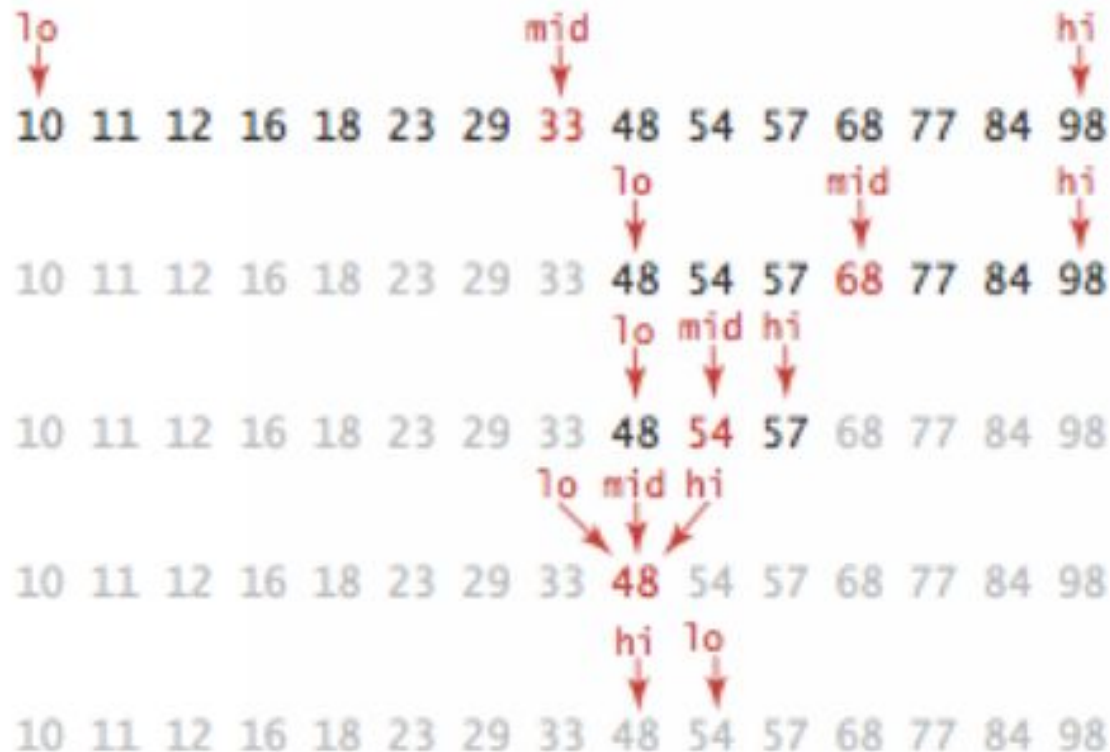
```
ALGORITHM BinarySearch( $A[0..n - 1]$ ,  $K$ )  
    //Implements nonrecursive binary search  
    //Input: An array  $A[0..n - 1]$  sorted in ascending order and  
    //      a search key  $K$   
    //Output: An index of the array's element that is equal to  $K$   
    //      or  $-1$  if there is no such element  
     $l \leftarrow 0$ ;  $r \leftarrow n - 1$   
    while  $l \leq r$  do  
         $m \leftarrow \lfloor (l + r) / 2 \rfloor$   
        if  $K = A[m]$  return  $m$   
        else if  $K < A[m]$   $r \leftarrow m - 1$   
        else  $l \leftarrow m + 1$   
    return  $-1$ 
```

Working principle of Binary Search

successful search for 23



unsuccessful search for 50



Second Algorithm Example: Binary Search

6. Analyze the algorithm.

- How many iterations of the while loop is needed for the algorithm to solve the problem and return (or stop)?

or

- How many times the search key is compared with an element of the array (**if** comparisons)?

- **Worst case** (the item is not present in the sequence or it is only possible to find it during the last iteration, e.g., the element is the minimum or maximum):
 - **$\log n$ iterations of $2 \cdot \log n$ comparisons**
- **Best case** (the item is sitting just in the middle of the sequence)
 - **0 iterations or 1 comparisons.**
- **Average case** (the item can be anywhere) $\approx$ **$\log n$ iterations**
 - **depends on assumptions about the probabilistic distribution of input data, whether duplicates are allowed or not, etc.**

Interesting facts about binary search

- The first binary search algorithm was designed in 1946.
- It was not possible to design and write an error free binary search algorithm until 1962.

The sorting problem

Ex. Student records in a university.

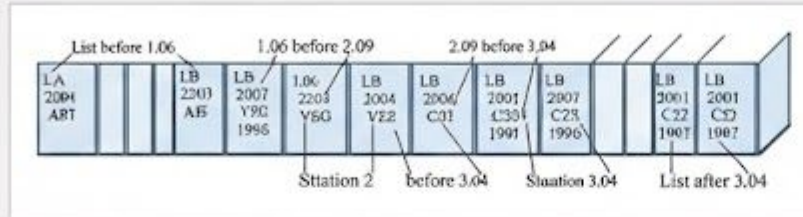
	Name	ID	Gender	Phone	Department
	Chen	3	A	614-470-0023	208 Math
	Rahde	2	A	212-741-8555	242 Physics
	Gazel	4	B	780-093-9872	101 Chemistry
item →	Fafla	1	A	760-693-5675	101 Biology
	Kanaga	3	B	604-112-8640	221 Biology
	Andrews	1	A	564-380-0022	607 Literature
key →	Baide	4	C	874-628-1212	121 Mathman

Sort. Rearrange array of N items into ascending order.

Name	ID	Gender	Phone	Department
Andrews	1	A	564-380-0022	607 Literature
Baide	4	C	874-628-1212	121 Mathman
Chen	3	A	614-470-0023	208 Math
Fafla	1	A	760-693-5675	101 Biology
Gazel	4	B	780-093-9872	101 Chemistry
Kanaga	3	B	604-112-8640	221 Biology
Rahde	2	A	212-741-8555	242 Physics

Sorting applications

Sorting applications



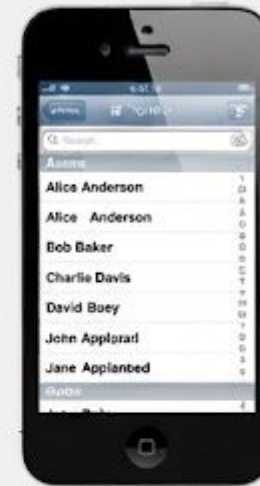
Library of Congress numbers



FedEx packages



playing cards



contacts



Hogwarts houses

Third Algorithm Example: Selection Sort

Same problem, same data structure, exact solution expected.
Here is the pseudocode for an algorithm to solve the problem via another sorting algorithm:

ALGORITHM *SelectionSort*($A[0..n - 1]$)

//Sorts a given array by selection sort

//Input: An array $A[0..n - 1]$ of orderable elements

//Output: Array $A[0..n - 1]$ sorted in nondecreasing order

for $i \leftarrow 0$ **to** $n - 2$ **do**

$min \leftarrow i$

for $j \leftarrow i + 1$ **to** $n - 1$ **do**

if $A[j] < A[min]$ $min \leftarrow j$

 swap $A[i]$ and $A[min]$

Working principle of Selection Sort

We want to sort the following sequence of numbers: 89, 45, 68, 90, 29, 34, 17

First iteration of outer loop		89	45	68	90	29	34	17
First iteration of outer loop	17		45	68	90	29	34	89
First iteration of outer loop	17	29		68	90	45	34	89
First iteration of outer loop	17	29	34		90	45	68	89
First iteration of outer loop	17	29	34	45		90	68	89
First iteration of outer loop	17	29	34	45	68		90	89
Result array when algorithm stops	17	29	34	45	68	89		90

Each line corresponds to one iteration of the algorithm, i.e., a pass through the list's tail to the right of the vertical bar; an element in bold indicates the smallest element found. Elements to the left of the vertical bar are in their final positions and are not considered in this and subsequent iterations.

Third Algorithm Example: Selection Sort

- **Worst case** (the given sequence of items are sorted in reverse order):
 - n^2 iterations of the inner for loop or n^2 comparisons
- **Best case** (the given sequence of items are already sorted in intended order)
 - n^2 iterations of the inner for loop or n^2 comparisons
- **Average case** (the given sequence of items are arbitrarily ordered)
 - n^2 iterations of the inner for loop or n^2 comparisons.

Thus, selection sort is a n^2 algorithm on all inputs

Fourth Algorithm Example: Bubble Sort

1. **Problem:** Given a sequence of items, rearrange them in-place in an increasing (ascending) sorted fashion.
2. **We want an exact solution.**
3. The appropriate data structure for this problem is **array**.
4. Here is the pseudocode for an algorithm to solve the problem:

ALGORITHM *BubbleSort*($A[0..n - 1]$)

//Sorts a given array by bubble sort

//Input: An array $A[0..n - 1]$ of orderable elements

//Output: Array $A[0..n - 1]$ sorted in nondecreasing order

for $i \leftarrow 0$ **to** $n - 2$ **do**

for $j \leftarrow 0$ **to** $n - 2 - i$ **do**

if $A[j + 1] < A[j]$ **swap** $A[j]$ and $A[j + 1]$

Working principle of Bubble Sort

We want to sort the following sequence of numbers: **89, 45, 68, 90, 29, 34, 17**

First iteration of outer loop	89	↔ [?]	45		68		90		29		34		17
	45		89	↔ [?]	68		90		29		34		17
	45		68		89	↔ [?]	90	↔ [?]	29		34		17
	45		68		89		29		90	↔ [?]	34		17
	45		68		89		29		34		90	↔ [?]	17
	45		68		89		29		34		17		90
Second iteration of outer loop	45	↔ [?]	68	↔ [?]	89	↔ [?]	29		34		17		90
	45		68		29		89	↔ [?]	34		17		90
	45		68		29		34		89	↔ [?]	17		90
	45		68		29		34		17		89		90
Third iteration	etc.												

A new line is shown after a swap of two elements is done. The elements to the right of the vertical bar are in their final positions and are not considered in subsequent iterations of the algorithm.

Fourth Algorithm Example: Bubble Sort

6. Analyze the algorithm.

- How many iterations of the inner for loop is needed for the algorithm to solve the problem and return (or stop)?

or

- How many comparison operations (**if** statements) must be executed for the algorithm to solve the problem and return (or stop)?

- **Worst case** (the given sequence of items are sorted in reverse order):
 - n^2 iterations of the inner for loop or n^2 comparisons
- **Best case** (the given sequence of items are already sorted in intended order)
 - n^2 iterations of the inner for loop or n^2 comparisons
- **Average case** (the given sequence of items are arbitrarily ordered)
 - n^2 iterations of the inner for loop or n^2 comparisons.

Bubble Sort: Improved Version

ALGORITHM *BubbleSort*($A[0..n-1]$)

// Sorts a given sequence of values by bubble sort.

//Input: An array $A[0..n - 1]$ of orderable elements

//Output: Array $A[0..n - 1]$ sorted in nondecreasing order

for $i \leftarrow 0$ **to** $n-2$ **do**

$swapped \leftarrow false$

for $j \leftarrow 0$ **to** $n-2-i$ **do**

if ($A[j+1] < A[j]$) **then**

 swap $A[j]$ and $A[j+1]$

$swapped \leftarrow true$

#if no item was swapped that means sorted.

if not($swapped$) **return**

- **Worst case :**
 - n^2 iterations of the inner for loop or n^2 comparisons
- **Best case**
 - n iterations of the inner for loop or n comparisons
- **Average case** (depends ...) $\approx n^2$ iterations or comparisons

Sorting Algorithm Visualizer

A nice web site that helps understand how several sorting algorithms work

Have a look!

[Interactive Sorting Algorithm Visualiser](#)

Analysis of the complexity of Algorithms

- Issues in algorithm design:

- correctness
- time efficiency
- space efficiency
- optimality

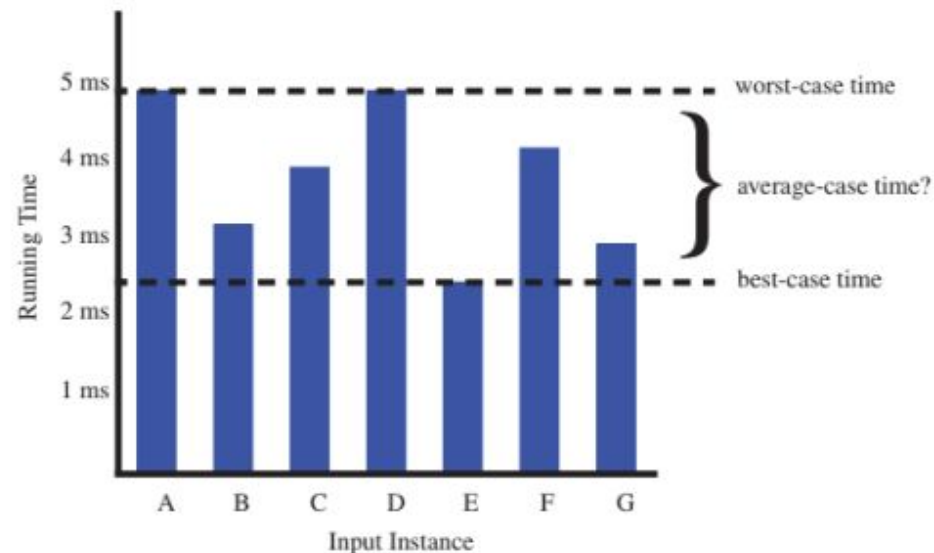
- Approaches:

- theoretical analysis
- empirical analysis

Best-case, average-case, worst-case

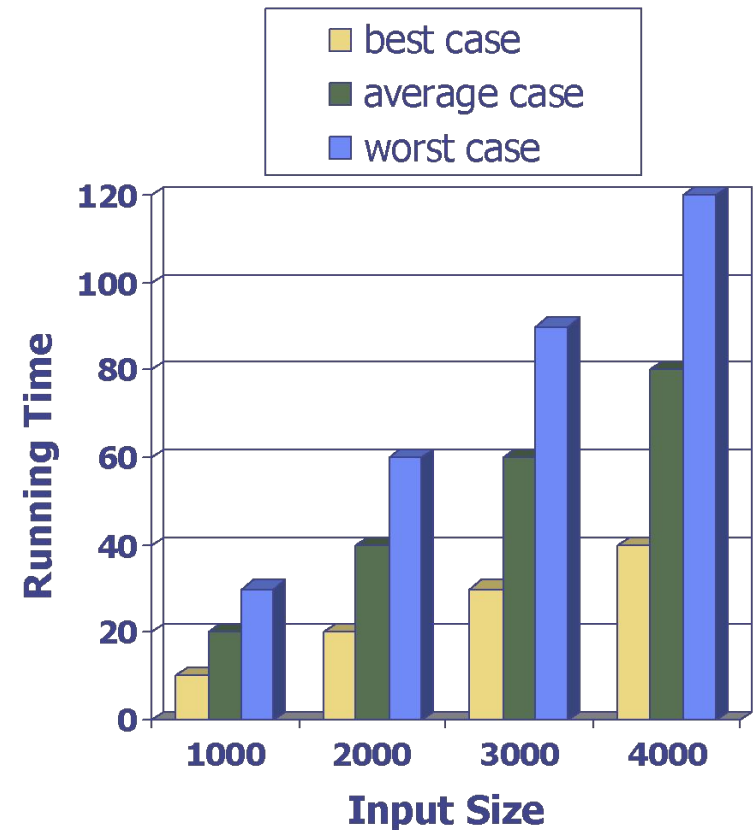
For some algorithms efficiency depends on form of input:

- **Worst case:** $C_{\text{worst}}(n)$ – maximum over inputs of size n
- **Best case:** $C_{\text{best}}(n)$ – minimum over inputs of size n
- **Average case:** $C_{\text{avg}}(n)$ – “average” over inputs of size n
 - Number of times the basic operation will be executed on typical input
 - NOT the average of worst and best case
 - Expected number of basic operations considered as a random variable under some assumption about the probability distribution of all possible inputs.



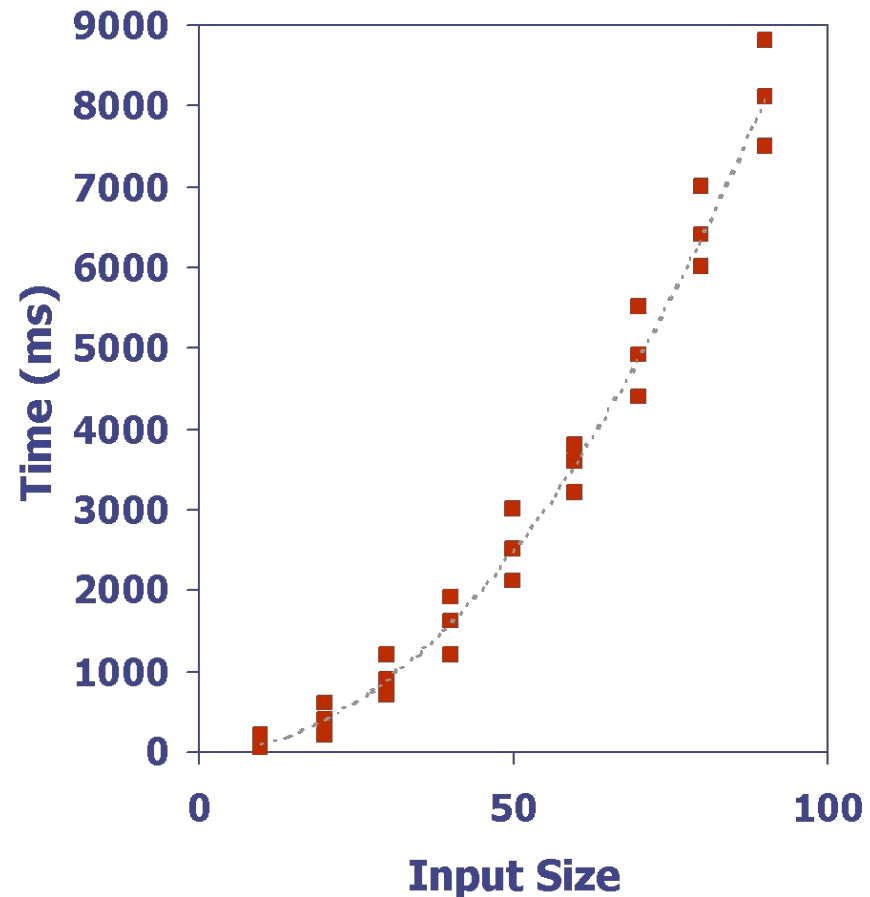
Running Time

- Most algorithms transform input objects into output objects.
- The running time of an algorithm typically grows with the input size.
- Average case time is often difficult to determine.
- We focus on the worst case running time.
 - Easier to analyze
 - Crucial to applications such as games, finance and robotics



Empirical/Experimental Analysis

- Write a program implementing the algorithm
- Run the program with inputs of varying size and composition
- Use a function, like the built-in `clock()` function, to get an accurate measure of the actual running time
- Plot the results



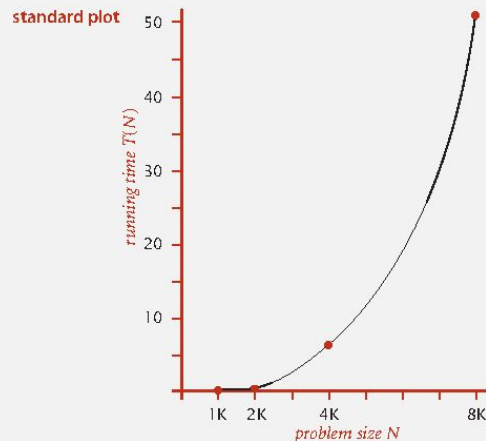
Analysis

We can plot the running time of our program as a function of the input size.

We can try to fit a line on the **logarithmic** axis by making the same drawing on a **logarithmic** scale.

Data analysis

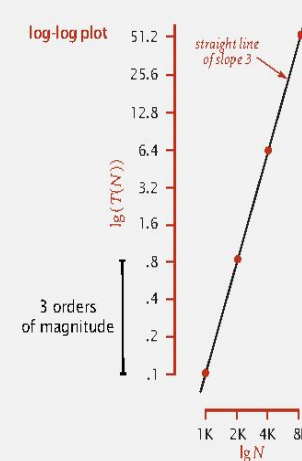
Standard plot. Plot running time $T(N)$ vs. input size N .



17

Data analysis

Log-log plot. Plot running time $T(N)$ vs. input size N using **log-log scale**.



$$\lg(T(N)) = b \lg N + c$$
$$b = 2.999$$
$$c = -33.2103$$

$$T(N) = a N^b, \text{ where } a = 2^c$$

Regression. Fit straight line through data points: $a N^b$.

Hypothesis. The running time is about $1.006 \times 10^{-10} \times N^{2.999}$ seconds.

18

Deficiencies of empirical analysis

- It is difficult to make precise measurements.
- The results are very dependent on the hardware our program is running on, the operating system, compiler, interpreter, etc., and other software elements running in the same environment.
- For example, the measurements in the table below were taken on a Macbook Pro 2.2GHz 2GB RAM running OS X.

Cost of basic operations

Challenge. How to estimate constants.

operation	example	nanoseconds †
integer add	<code>a + b</code>	2.1
integer multiply	<code>a * b</code>	2.4
integer divide	<code>a / b</code>	5.4
floating-point add	<code>a + b</code>	4.6
floating-point multiply	<code>a * b</code>	4.2
floating-point divide	<code>a / b</code>	13.5
sine	<code>Math.sin(theta)</code>	91.3
arctangent	<code>Math.atan2(y, x)</code>	129.0
...	...	...

† Running OS X on Macbook Pro 2.2GHz with 2GB RAM

Limitations of Experiments

- It is necessary to implement the algorithm, which may be difficult
- Results may not be indicative of the running time on other inputs not included in the experiment.
- In order to compare two algorithms, the same hardware and software environments must be used

Theoretical Analysis

- Uses a high-level description of the algorithm instead of an implementation;
Pseudocode
- Characterizes running time as a function of the input size, n .
- Takes into account all possible inputs
- Allows us to evaluate the speed of an algorithm independent of the hardware/software environment

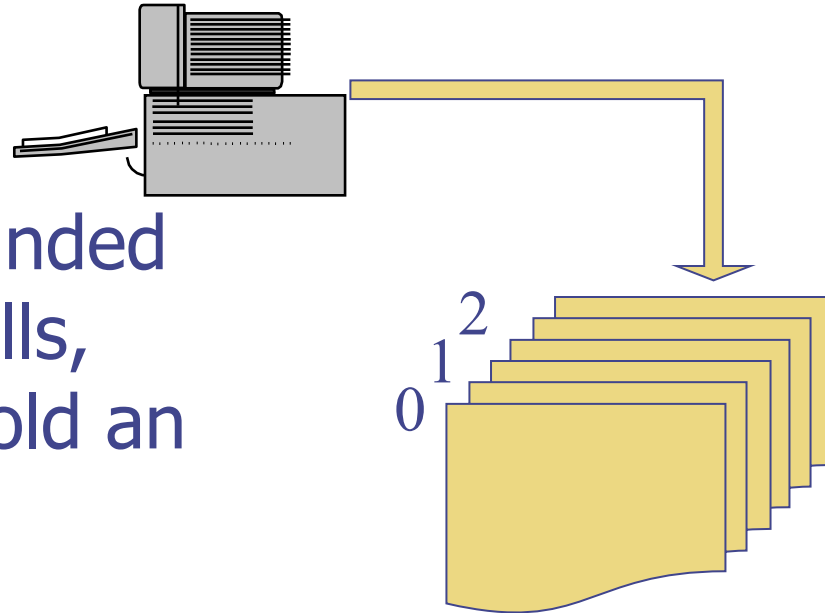
Pseudocode Details

- Control flow
 - if ... then ... [else ...]
 - while ... do ...
 - repeat ... until ...
 - for ... do ...
 - Indentation replaces braces
- Method declaration

Algorithm *method* (*arg* [, *arg*...])
Input ...
Output ...
- Method/Function call
var.method (*arg* [, *arg*...])
- Return value
return *expression*
- Expressions
 - ← Assignment
(like = in C++)
 - = Equality testing
(like == in C++)
 - n*² Superscripts and other
mathematical formatting
allowed

The Random Access Machine (RAM) Model

- CPU
- A CPU
- An potentially unbounded bank of **memory** cells, each of which can hold an arbitrary number or character
- Memory cells are numbered and accessing any cell in memory takes unit time.



Basic/Primitive Operations

- Basic computations performed by an algorithm
 - Identifiable in pseudocode
 - Largely independent from the programming language
 - Exact definition not important (we will see why later)
 - Assumed to take a constant amount of time in the RAM model
- Examples:
 - Evaluating an expression
 - Assigning a value to a variable
 - Indexing into an array
 - Calling a method
 - Returning from a method
 - A jump (goto)

Counting Primitive Operations

- By inspecting the pseudocode, we can determine the maximum number of primitive operations executed by an algorithm, as a function of the input size

Algorithm <i>arrayMax</i> (<i>A</i> , <i>n</i>)	# operations
<i>currentMax</i> $\leftarrow A[0]$	2 (1 array index, 1 assignment)
for <i>i</i> $\leftarrow 1$ to <i>n</i> - 1 do	<i>n</i> + 1 (1 assignment, <i>n</i> -1 comparisons, 1 jump)
if <i>A</i> [<i>i</i>] > <i>currentMax</i> then	2(<i>n</i> - 1) (1 array index, 1 conditional)
<i>currentMax</i> $\leftarrow A[i]$	2(<i>n</i> - 1) (1 array index, 1 assignment)
<i>{increment counter i (i ← i+1)}</i>	2(<i>n</i> - 1) (1 addition, 1 assignment)
return <i>currentMax</i>	1 (1 return)
Total $7n - 2$	

*Note that the for loop above is iterated *n*-1 times, as *i* takes values from 1 to *n*-1, inclusive.*

*The **for** statement itself is visited *n*-1 times, with a single assignment (*i* ← 1) during first visit, and one comparison (*i* ≤ *n*-1) during each visit, plus one jump out of the loop at the end. Hence;*

$$1 + n - 1 + 1 = n + 1 \text{ (the increment of loop counter given separately)}$$

Estimating Running Time

- Algorithm *arrayMax* executes $7n - 1$ primitive operations in the worst case. Define:
 - a = Time taken by the fastest primitive operation
 - b = Time taken by the slowest primitive operation
- Let $T(n)$ be worst-case time of *arrayMax*. Then
$$a(7n - 2) \leq T(n) \leq b(7n - 2)$$
- Hence, the running time $T(n)$ is bounded by two linear functions

Types of formulas for basic operation's count

- Exact formula, such as $7n - 2$, i.e.;

$$\begin{aligned} \text{e.g., } T(n) &= n(n-1)/2 \quad \text{or} \quad (n-1)/2, \text{ etc.} \\ &= \frac{1}{2}n^2 - \frac{1}{2}n \quad \text{or} \quad \frac{1}{2}n - \frac{1}{2} \end{aligned}$$

- Formula indicating order of growth with specific multiplicative constant:

$$\begin{aligned} \text{e.g., } T(n) &= \frac{1}{2}n^2 - \frac{1}{2}n \approx \frac{1}{2}n^2 = \mathbf{0.5 \, n^2} \\ T(n) &= \frac{1}{2}n - \frac{1}{2} \approx \frac{1}{2}n = \mathbf{0.5 \, n} \end{aligned}$$

- Formula indicating order of growth with unknown multiplicative constant:

$$\text{e.g., } T(n) \approx cn^2$$

Order of growth of Running Time

- **Remember:** we are interested in question like:
 - *How much faster would this algorithm run on a machine that is 10 times faster than the one we have?*
 - *how much longer will the algorithm run if we double its input size?*
- **Most important:** Order of growth within a constant multiple as $n \rightarrow \infty$
- Changing the hardware/ software environment:
 - Affects $T(n)$ by a constant factor, but
 - Does **not** alter the growth rate of $T(n)$
- The **growth rate** of the running time $T(n)$ is an intrinsic property of the algorithm itself.

“Order of Growth” Classification

- Two functions f and g have the relation $f(n) \sim g(n)$, *meaning that f and g grow with the same rate*, if;
 - $\lim_{n \rightarrow \infty} f(n)/g(n) = \langle A \text{ constant value} \rangle$
- **Definition:**
 - If there is a positive constant so that the following equation holds:
 - $f(n) \sim c * g(n)$
 - Then the order of growth of the two functions ***f and g are the same.***
 - or, in other words:
 - The order of growth classification of ***$f(n)$ is $g(n)$.***
- “Order of growth” definition ignores leading coefficients and lower order terms in a function.

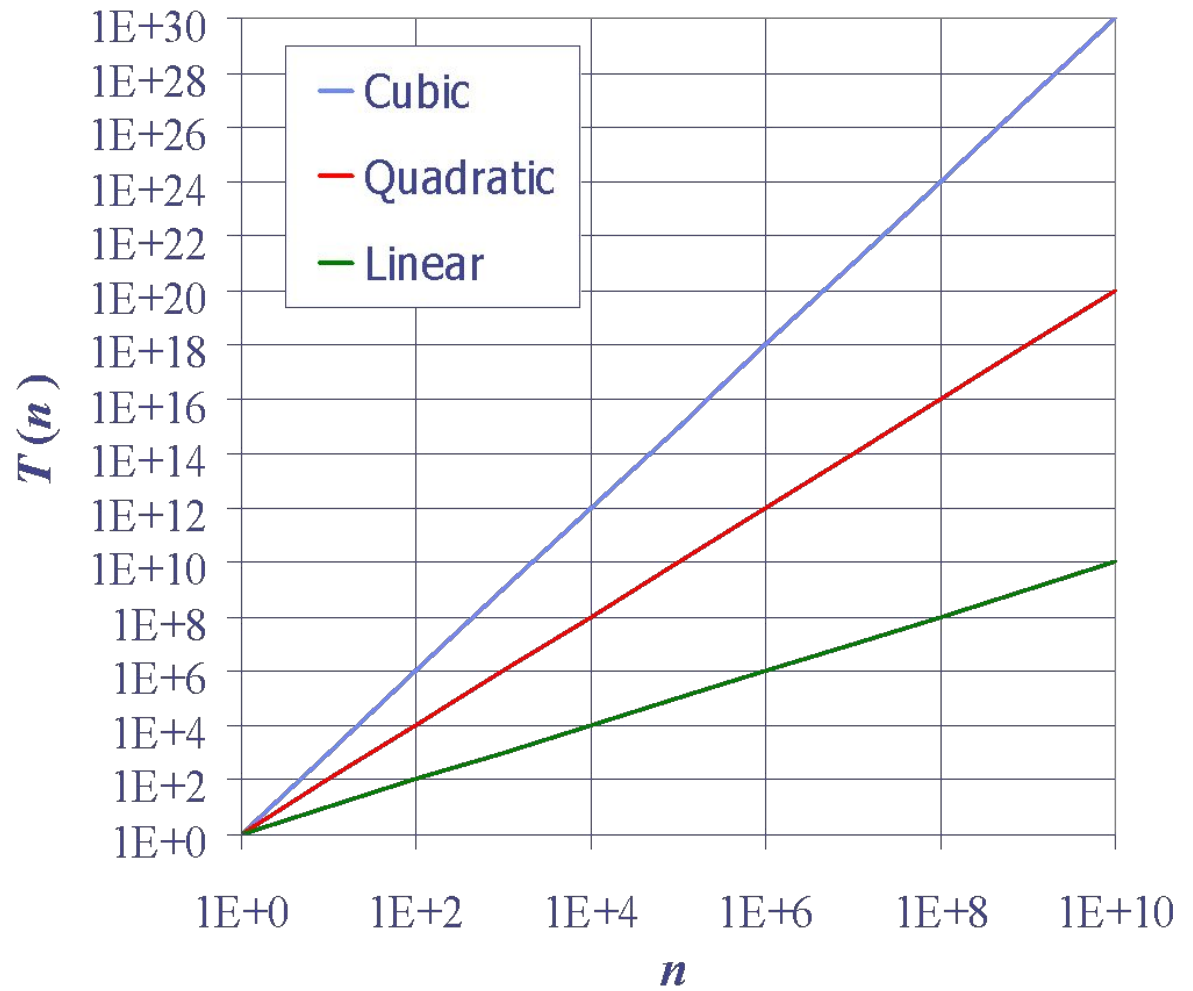
Values of some important functions as $n \rightarrow \infty$

n	$\log_2 n$	n	$n \log_2 n$	n^2	n^3	2^n	$n!$
10	3.3	10^1	$3.3 \cdot 10^1$	10^2	10^3	10^3	$3.6 \cdot 10^6$
10^2	6.6	10^2	$6.6 \cdot 10^2$	10^4	10^6	$1.3 \cdot 10^{30}$	$9.3 \cdot 10^{157}$
10^3	10	10^3	$1.0 \cdot 10^4$	10^6	10^9		
10^4	13	10^4	$1.3 \cdot 10^5$	10^8	10^{12}		
10^5	17	10^5	$1.7 \cdot 10^6$	10^{10}	10^{15}		
10^6	20	10^6	$2.0 \cdot 10^7$	10^{12}	10^{18}		

Values (some approximate) of several functions important for analysis of algorithms

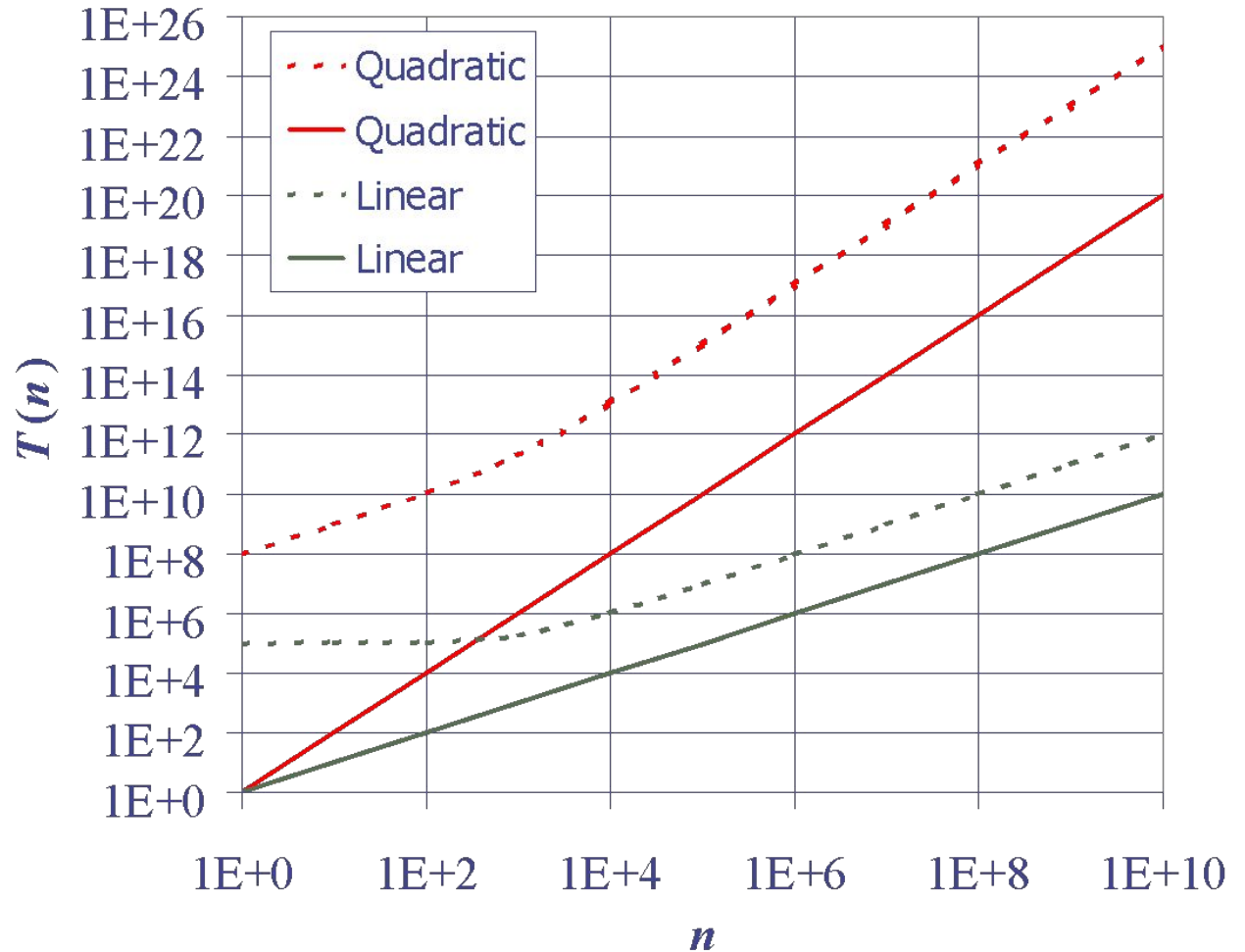
Growth Rates

- Growth rates of functions:
 - Linear $\approx n$
 - Quadratic $\approx n^2$
 - Cubic $\approx n^3$
- In a log-log chart, the slope of the line corresponds to the growth rate of the function



Constant Factors

- The growth rate is not affected by
 - constant factors or
 - lower-order terms
- Examples
 - $10n + 10^5$ is a linear function
 - $10^5 n^2 + 10^8 n$ is a quadratic function



Big-Oh Notation

- Given functions $f(n)$ and $g(n)$, we say that

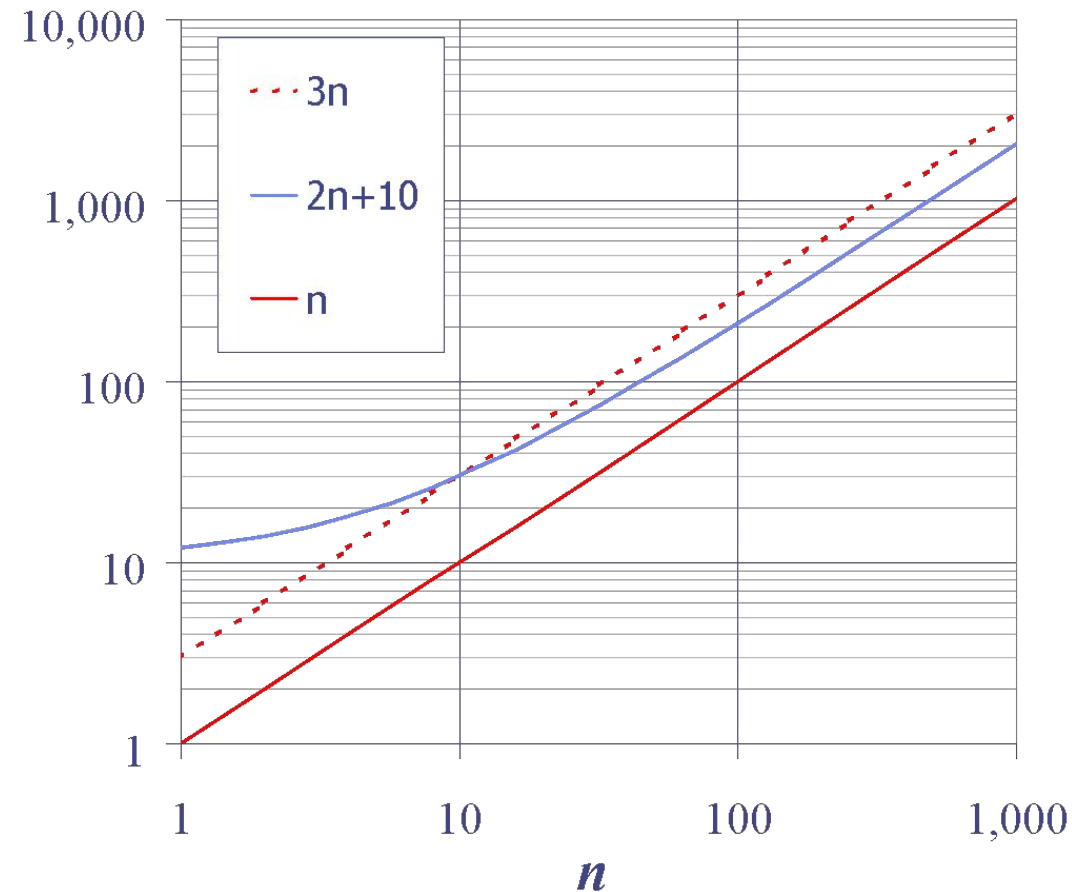
$f(n)$ is $O(g(n))$

if there are positive constants c and n_0 such that

$$f(n) \leq cg(n) \text{ for } n \geq n_0$$

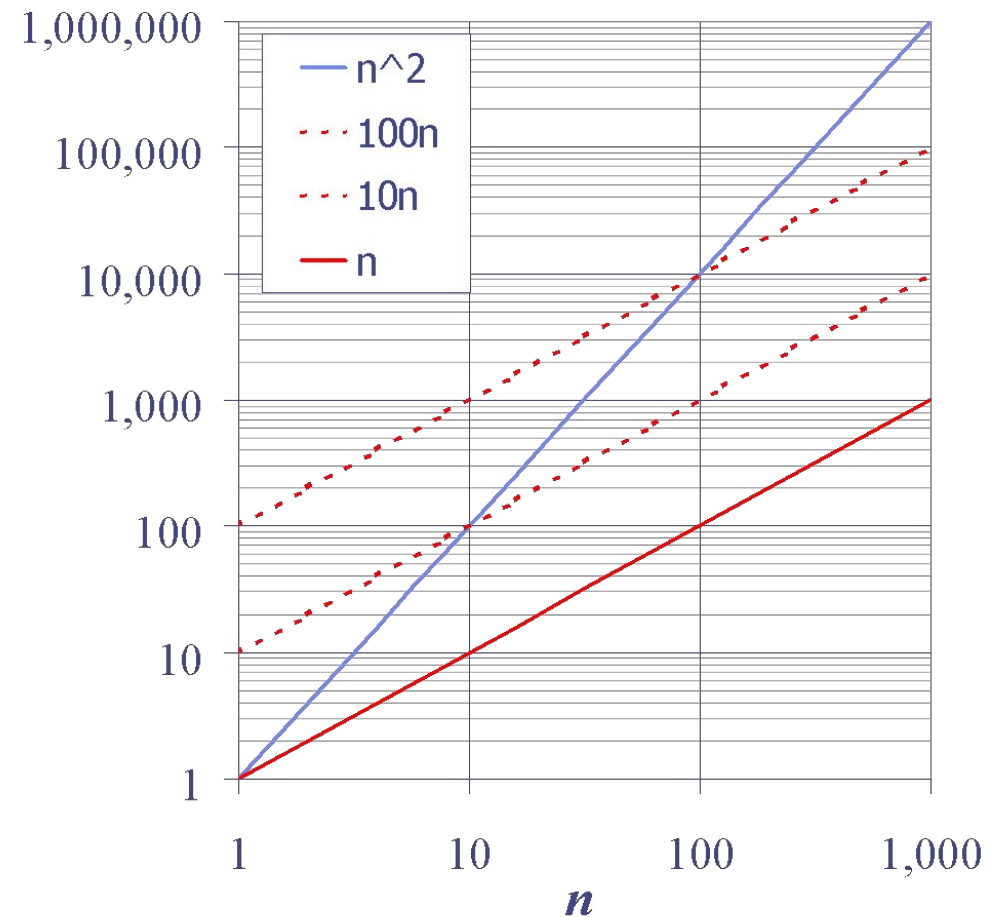
- Example: $2n + 10$ is $O(n)$

- $2n + 10 \leq cn$
- $(c - 2)n \geq 10$
- $n \geq 10/(c - 2)$
- Pick $c = 3$ and $n_0 = 10$



Big-Oh Example

- Example: the function n^2 is **not** $O(n)$
 - $n^2 \leq cn$
 - $n \leq c$
 - The above inequality cannot be satisfied since c must be a constant



More Big-Oh Examples

■ $7n-2$

$7n-2$ is $O(n)$

need $c > 0$ and $n_0 \geq 1$ such that $7n-2 \leq c \cdot n$ for $n \geq n_0$

this is true for $c = 7$ and $n_0 = 1$

■ $3n^3 + 20n^2 + 5$

$3n^3 + 20n^2 + 5$ is $O(n^3)$

need $c > 0$ and $n_0 \geq 1$ such that $3n^3 + 20n^2 + 5 \leq c \cdot n^3$ for $n \geq n_0$

this is true for $c = 4$ and $n_0 = 21$

■ $3 \log n + \log \log n$

$3 \log n + \log \log n$ is $O(\log n)$

need $c > 0$ and $n_0 \geq 1$ such that $3 \log n + \log \log n \leq c \cdot \log n$ for $n \geq n_0$

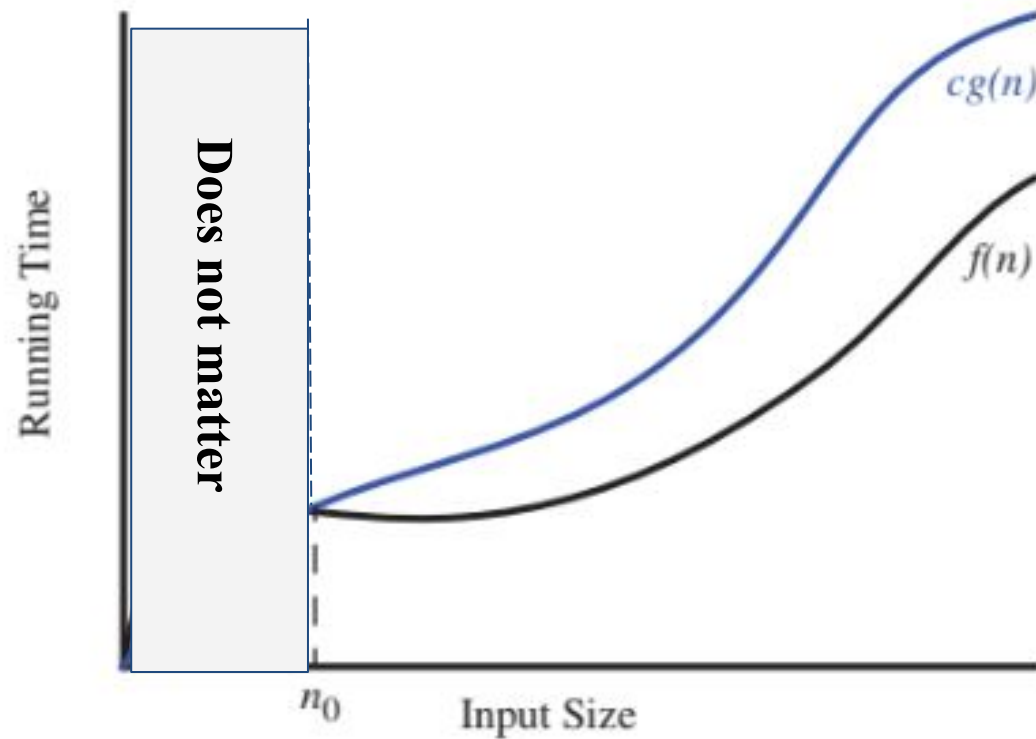
this is true for $c = 4$ and $n_0 = 2$

Big-Oh and Growth Rate

- The big-Oh notation gives an upper bound on the growth rate of a function
- The statement " $f(n)$ is $O(g(n))$ " means that the growth rate of $f(n)$ is no more than the growth rate of $g(n)$
- We can use the big-Oh notation to rank functions according to their growth rate

	$f(n)$ is $O(g(n))$	$g(n)$ is $O(f(n))$
$g(n)$ grows more	Yes	No
$f(n)$ grows more	No	Yes
Same growth	Yes	Yes

Big-oh (O)



The “big-Oh” notation. The function $f(n)$ is $O(g(n))$, since $f(n) \leq c \cdot g(n)$ when $n \geq n_0$.

Big-Oh Rules

- If $f(n)$ is a polynomial of degree d , then $f(n)$ is $O(n^d)$, i.e.,
 1. Drop lower-order terms
 2. Drop constant factors
- Use the "smallest possible" class of functions
 - Say " $2n$ is $O(n)$ " instead of " $2n$ is $O(n^2)$ "
- Use the simplest expression of the class
 - Say " $3n + 5$ is $O(n)$ " instead of " $3n + 5$ is $O(3n)$ "

Asymptotic Algorithm Analysis

- The asymptotic analysis of an algorithm determines the running time in big-Oh notation
- To perform the asymptotic analysis
 - We find the number of primitive operations executed as a function of the input size (worst, best, average)
 - We express this function with big-Oh notation
- Example:
 - We determine that algorithm *arrayMax* executes at most $7n - 1$ primitive operations
 - We say that algorithm *arrayMax* "runs in $O(n)$ time"
- Since constant factors and lower-order terms are eventually dropped anyhow, we can disregard them when counting primitive operations

Basic asymptotic efficiency classes

1	constant
$\log n$	logarithmic
n	linear
$n \log n$	n-log-n or linearithmic
n^2	quadratic
n^3	cubic
2^n	exponential
$n!$	factorial
n^n	?????